



武汉大学
Wuhan University

第三章 Verilog HDL 基本语法

武汉大学计算机学院 2025-2026第一学期



主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

数据类型

运算符

基本语句

模块化程序设计

四、循环语句

1. forever循环

格式(1): **forever** 语句;

格式(2): **forever**
 begin
 多条语句;
 end

- **forever**是一种**无穷循环**控制语句，它不断地执行其后的语句或语句块
- **forever**语句要写在**initial**块中
- **forever**语句常用来产生周期性的波形，作为**仿真激励**信号

例如:

```
initial  
begin  
    clk = 1'b0;  
    forever #10 clk = ~clk;  
end
```

功能：每隔10个时间单位，clk反向，时钟周期为20个时间单位



四、循环语句

2. repeat循环

格式(1): **repeat** (循环次数表达式) 语句;

格式(2) : **repeat** (循环次数表达式)

begin

语句块;

end

repeat语句执行**指定循环次数**，如果循环次数表达式的值不确定，则循环次数为0。

四、循环语句

3. while循环

格式：**while**（条件）

begin

语句块；

end

- **while**语句实现的是一种**条件循环**，只有在指定的循环条件为真时才会重复执行循环体
- 如果条件表达式的值在开始时不为真，那么过程语句将永远不会被执行

四、循环语句

4. for循环

格式：

for(循环变量=**初值**; 循环变量< **终值**; 循环变量=循环变量+**步长值**)

begin

语句块;

end

for语句重复执行的次数由语句中的参数确定, 即

循环重复次数= (终值-初值) / 步长值

3.4 Verilog HDL的基本语句

四、循环语句

用for语句描述七人投票表决器

```
module voter7(pass,vote);  
    output pass;  
    input[6:0] vote; //投票值, 1表示赞成, 0表示反对  
    reg[2:0] sum;  
    integer i; //循环变量  
    reg pass;  
    always @(vote)  
        begin  
            sum=0;  
            for(i=0;i<=6;i=i+1)  
                if(vote[i]) sum=sum+1; //统计投赞成票的人数  
            if(sum[2]) pass=1; //超过4人赞成, 则通过  
            else pass=0;  
        end  
endmodule
```

sum[2:0]

1		
---	--	--

四、循环语句

四种循环语句的比较

repeat: 将一块语句循环执行**确定次数**

repeat (次数表达式) <语句>

while: 在条件**表达式为真时**一直循环执行

while (条件表达式) <语句>

forever: 重复执行**直到仿真结束**

forever <语句>

for: 在执行过程中对变量进行计算和判断, 在**条件满足时**执行

for(循环变量赋初值; 条件表达式; 循环变量更新计算) <语句>

循环次数不确定的循环语句一般不能被综合。

五、任务与函数

- 任务（task）和函数（function）都用来实现程序中**相对独立**的某项功能，可被模块或其它任务及函数**调用**
- 把大的程序模块**分解**成较小的任务或函数，或把需要**重复使用**的程序代码写出任务或函数多次调用，可使程序条理清晰易懂，便于调试

◆二者的区别

函数有返回值，任务没有返回值

五、任务与函数

➤ 任务 (task) 定义格式

task 任务名

端口与类型说明

局部变量说明

块语句

endtask

任务调用格式:

任务名 (端口列表);

- 任务调用时和定义时的端口变量应是一一对应的
- 一个任务可以调用别的任务和函数，可以调用的任务和函数个数不限

五、任务与函数

```
module add16(a, b, cin, sum, cout);  
    input [15:0] a, b;  
    input cin;  
    output [15:0] sum;  
    output cout;  
    reg [15:0] sum;  
    reg cout;  
    reg c8;  
    always @(a or b or cin)  
        begin  
            adder8(a[7:0], b[7:0], cin, sum[7:0], c8);  
            adder8(a[15:8], b[15:8], c8, sum[15:8],  
cout);  
        end
```

```
task adder8;  
    input [7:0] ta, tb;  
    input tcin;  
    output [7:0] tsum;  
    output tcout;  
    begin  
        {tcout, tsum}=ta+tb+tcin;  
    end  
endtask  
  
endmodule
```

task adder8定义一个8位加法器。
过程块**always**两次调用**adder8**可实现16位加法。

五、任务与函数

➤ 函数的定义格式:

function <返回值位宽或类型说明> 函数名;

参数声明;

局部变量定义;

其它语句;

endfunction

➤ <返回值位宽或类型说明>是一个可选项，如果缺省，则返回值为1位寄存器类型的数据

函数调用格式:

变量名=函数名(输入参数列表);

➤ 函数的定义与调用须在一个module模块内

➤ 函数必须至少有一个输入参数, 输出变量由函数名担任

五、任务与函数

```
module example(a, b, c, d);  
    input [7:0] a, b;  
    output [2:0] c, d;  
  
    always @(a or b)  
        begin  
            c=get0(a);  
            d=get0(b);  
        end
```

功能： 分别统计8位二进制数a和b中0的位数。

```
function [2:0] get0;  
    input[7:0] x;  
    reg[2:0] count;  
    integer i;  
    begin  
        count=0;  
        for (i=0;i<=7;i=i+1)  
            if(x[i]==1'b0) count=count+1;  
        //循环检查输入数据x的每一位，计算出x中  
        //0的位数，并返回一个适当的值  
        get0=count; //输出变量中0的位数  
                    //输出变量与函数名相同  
    end  
endfunction  
endmodule
```

主要内容

Main Content

Quartus II 使用说明

Verilog HDL程序基本结构

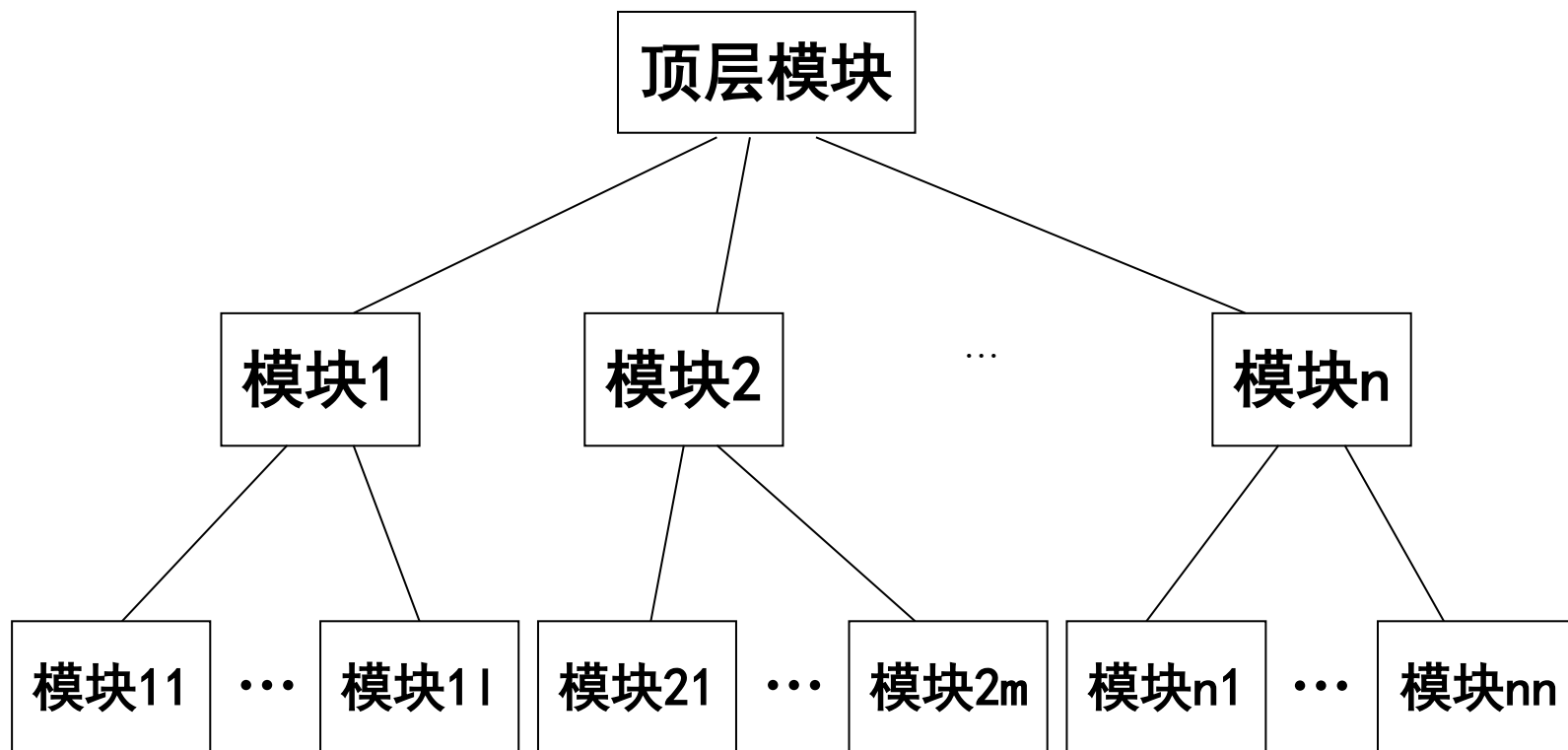
数据类型

运算符

基本语句

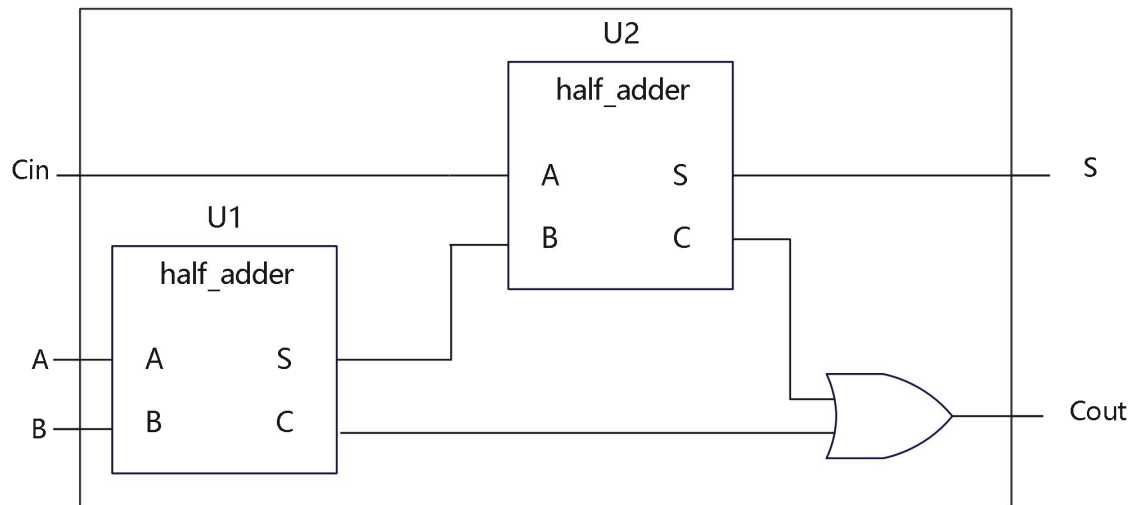
模块化程序设计

3.5 模块化程序设计



多模块之间的关系图

例3.5.1 引用半加器模块构成1位全加器



- 半加器不考虑低位来的进位，是2个1位二进制数相加
- 全加器考虑低位来的进位，是3个1位二进制数相加
- 引用半加器来构成全加器，需要引用半加器2次

```
moudle half_adder(A,B,S,C); //半加器模块
    input A,B;
    output S,C;
    assign S = A ^ B;
    assign C = A & B;
endmoudle

moudle full_adder(A,B,Cin,S,Co); //全加器模块
    input A,B,Cin;
    output S,Co;
    wire C1,C2,S1;
    half_adder u1(A,B,S1,C1);
    half_adder u2(Cin,S1,S,C2);
    assign Co = C1 | C2;
endmoudle
```



武汉大学
Wuhan University

第四章 组合逻辑电路

- 4.1 组合逻辑电路概述
- 4.2 组合逻辑电路分析
- 4.3 组合逻辑电路设计

武汉大学计算机学院 2025-2026秋季学期



主要内容

Main Content

组合逻辑电路概述

组合逻辑电路分析

组合逻辑电路设计

4.1 组合逻辑电路概述

数字逻辑电路 { 组合逻辑电路
时序逻辑电路

组合电路特点 { 功能上: 输出仅与该时刻的输入有关
结构上: 由门电路组成

主要内容

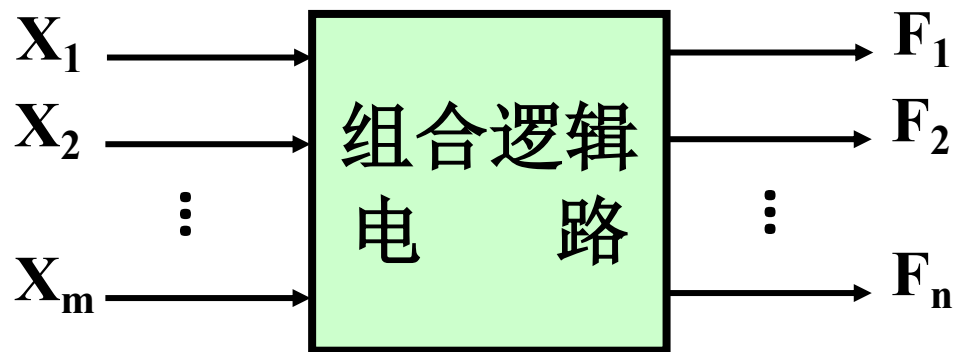
1. 组合电路的分析、设计方法
2. 常用集成组合逻辑电路

由小规模集成
电路组成

由中规模集成
电路组成

4.1 组合逻辑电路概述

组合逻辑电路： 输出跟随输入的变化



输入: $X_1, X_2, \dots, X_m$

输出: $F_1, F_2, \dots, F_n$

组合电路的特点:

- 电路中不包含记忆元件
- 输出无反馈到输入的回路
- 任意时刻, 电路的输出仅取决于当时的输入

逻辑关系:

$$\begin{cases} F_1 = f_1(X_1, X_2, \dots, X_m) \\ F_2 = f_2(X_1, X_2, \dots, X_m) \\ F_n = f_n(X_1, X_2, \dots, X_m) \end{cases}$$

主要内容

Main Content

组合逻辑电路概述

组合逻辑电路分析

组合逻辑电路设计

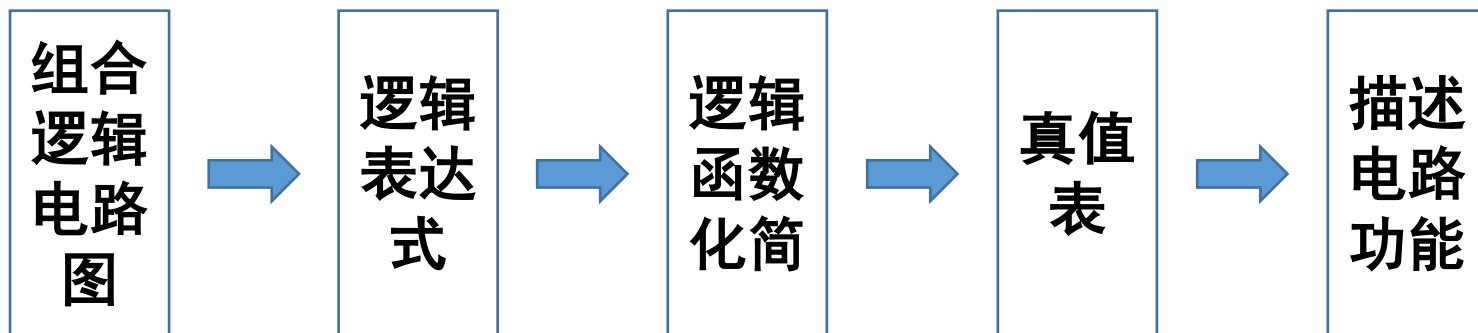
4.2 组合逻辑电路分析

■ 分析目的

找出电路的输出与输入变量的逻辑关系, 确定电路所实现的逻辑功能

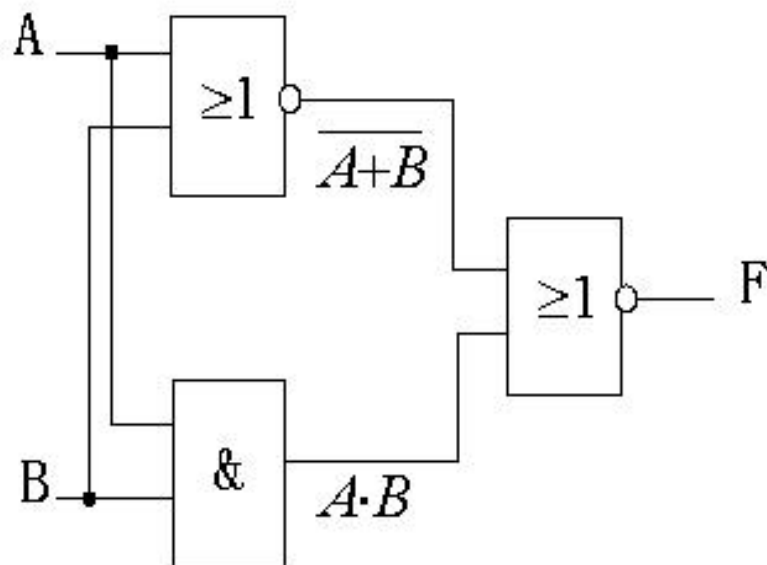
■ 分析方法

- 从输入级开始, 逐级写出门的逻辑表达式
- 对表达式进行化简
- 列真值表
- 描述电路的逻辑功能



4.2 组合逻辑电路分析

例1：试分析下图所示逻辑电路的功能。



组合逻辑电路的级数：
输入信号从输入端到输出端所经历的逻辑门的最大数目

解：该电路为二级组合电路。

(1) 写出电路的逻辑表达式

$$\begin{aligned}
 F &= \overline{\overline{A + B + AB}} \\
 &= (A + B)(\overline{A} + \overline{B}) \\
 &= \overline{A}B + A\overline{B}
 \end{aligned}$$

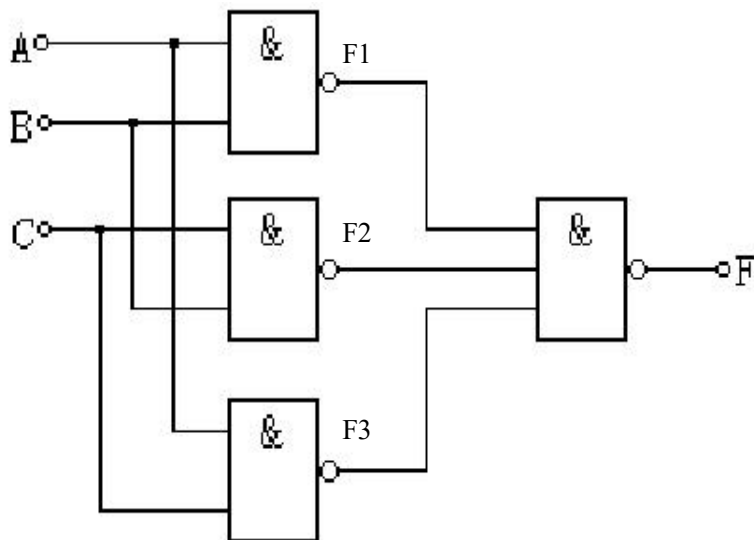
反演律
分配律

(2) 描述电路的逻辑功能

该函数表达式比较简单，不用列真值表，由表达式可知此电路是一个**异或电路**。

4.2 组合逻辑电路分析

例2：试分析图中所示逻辑电路的功能。



(2) 列真值表

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(1) 写出电路的逻辑表达式

$$F = \overline{F_1 \cdot F_2 \cdot F_3}$$

$$= \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}}$$

$$= AB + BC + AC$$

反演律

(3) 描述电路的逻辑功能

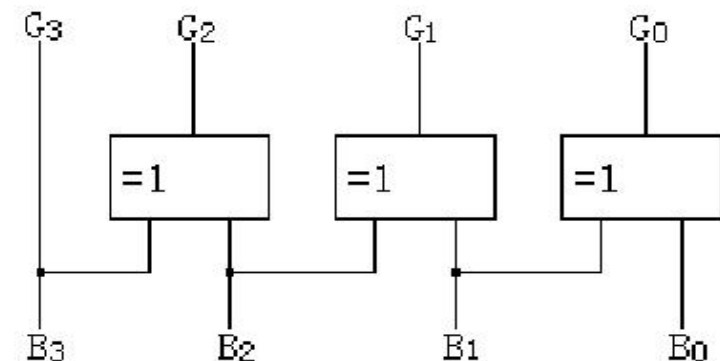
多数输入变量为1，输出F为1；

多数输入变量为0，输出F为0

因此，该电路为少数服从多数电路，也称多数表决电路。

4.2 组合逻辑电路分析

例3：试分析图中所示电路逻辑，写出表达式。



(1) 写出电路的逻辑表达式

$$\begin{cases} G_3 = B_3 \\ G_2 = B_3 \oplus B_2 \\ G_1 = B_2 \oplus B_1 \\ G_0 = B_1 \oplus B_0 \end{cases}$$

(2) 列写真值表

自然二进制码	格雷码
$B_3 B_2 B_1 B_0$	$G_3 G_2 G_1 G_0$
0 0 0 0	0 0 0 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	0 1 1 0
0 1 0 1	0 1 1 1
0 1 1 0	0 1 0 1
0 1 1 1	0 1 0 0
1 0 0 0	1 1 0 0
1 0 0 1	1 1 0 1
1 0 1 0	1 1 1 1
1 0 1 1	1 1 1 0
1 1 0 0	1 0 1 0
1 1 0 1	1 0 1 1
1 1 1 0	1 0 0 1
1 1 1 1	1 0 0 0

(3) 描述电路的逻辑功能

自然二进制码到格雷码的转换电路

主要内容

Main Content

组合逻辑电路概述

组合逻辑电路分析

组合逻辑电路设计

4.3 组合逻辑电路设计

■ 设计目标：

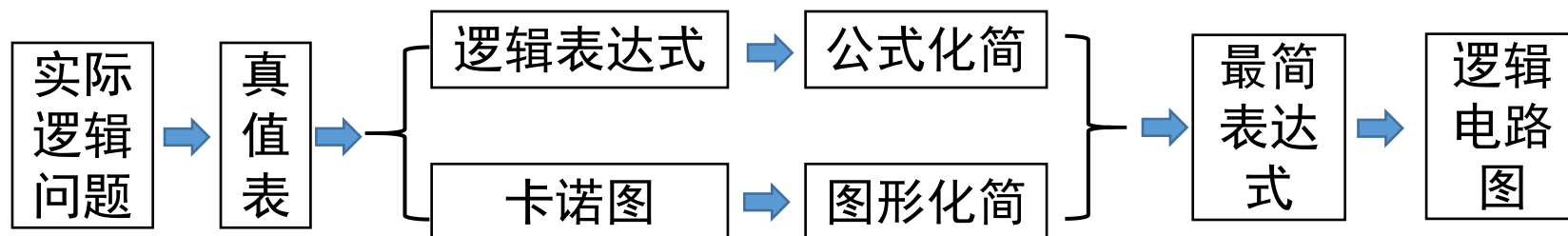
- 根据逻辑功能的要求，得到实现该功能的**最优**逻辑电路。

■ 最优标准：

- 小规模SSI — 逻辑门数最少，逻辑门级数最少，逻辑门之间的连线最少
- 中大规模MSI— 集成块数最少

■ 设计方法

- 传统方法——用逻辑函数实现



- 编程方法——用硬件描述语言编程实现

4.3 组合逻辑电路设计

例1 在举重比赛中，有两名副裁判，一名主裁判。当**两名以上裁判**（**必须包括主裁判在内**）认为运动员上举杠铃合格，按动电钮，裁决合格信号灯亮，则该运动员成绩有效。试设计该电路。

（一）传统方法

解：（1）确定输入输出

根据题意，有3个输入变量，1个输出变量。

➤ 设3个输入变量分别为A、B、C，其中**A表示主裁判**

输入：1表示同意，0表示不同意；

➤ 设输出变量为F

输出：1表示通过，0表示不通过

（2）列出真值表

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

4.3 组合逻辑电路设计

真值表

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(3) 由真值表写出表达式

$$Y = \bar{A}\bar{B}C + A\bar{B}\bar{C} + ABC$$

(4) 化简表达式

$$\begin{aligned} Y &= \bar{A}\bar{B}C + A\bar{B}\bar{C} + ABC \\ &= \bar{A}\bar{B}C + AB(\bar{C} + C) \\ &= \bar{A}\bar{B}C + AB \\ &= A(B + \bar{B}C) \\ &= A(B + C) = AB + AC \end{aligned}$$

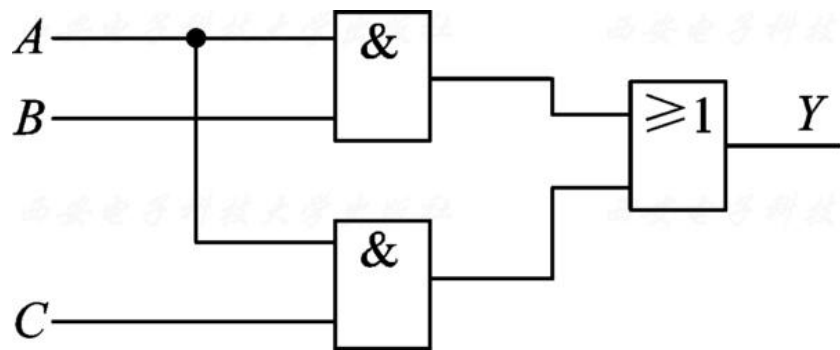
4.3 组合逻辑电路设计

$$Y=AB+AC$$

(5) 画出逻辑电路图

- 同一个电路可用不同的逻辑门来实现，需要根据所用的逻辑门对化简后的逻辑函数进行变形处理
- 除了基本逻辑门电路之外，常用的逻辑门电路有与非门、或非门、与或非门等

方法1.1 用与门、或门实现



4.3 组合逻辑电路设计

$$Y=AB+AC$$

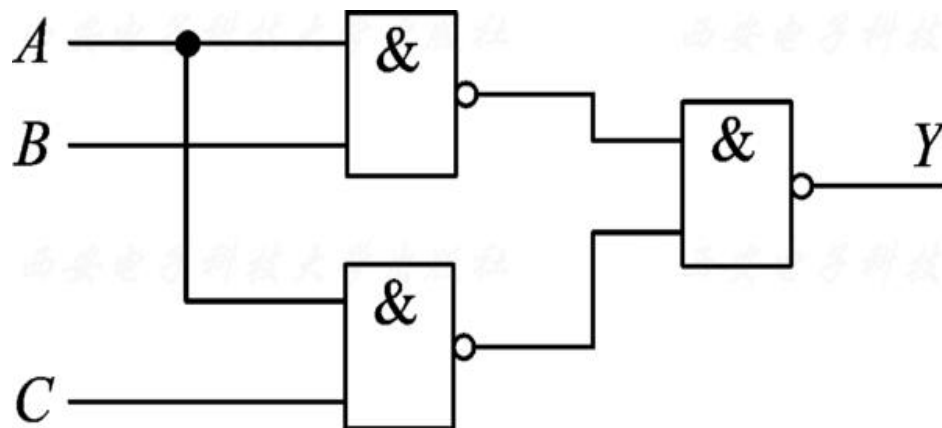
(5) 画出逻辑电路图

方法1.2 用与非门实现

与非-与非表达式:

$$Y = AB + AC$$

$$= \overline{\overline{AB + AC}} = \overline{\overline{AB} \cdot \overline{AC}}$$



4.3 组合逻辑电路设计

$$Y = AB + AC$$

(5) 画出逻辑电路图

方法1.3 用或非门实现

◆ 通过公式法求或非-或非表达式

$$\begin{aligned} Y &= AB + AC \\ &= \overline{\overline{AB} + \overline{AC}} = \overline{\overline{A(B+C)}} \\ &= \overline{\overline{A+B+C}} = \overline{\overline{A+B+C}} \end{aligned}$$

◆ 通过卡诺图求或非-或非表达式:

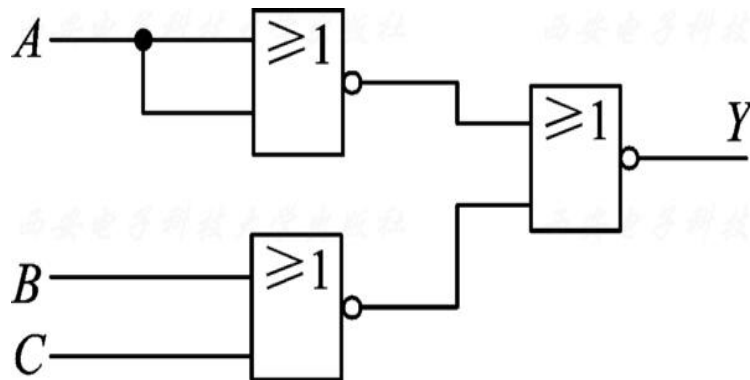
➤ 圈0, 得到或-与表达式

➤ 将或-与表达式两次求非

$$Y = (B + C) A$$

$$Y = \overline{\overline{(B + C) A}} = \overline{\overline{B + C} + \overline{A}}$$

	A	0	1
BC	00	0	0
	01	0	1
	11	0	1
	10	0	1



4.3 组合逻辑电路设计

(二) 编程方法

用Verilog HDL
行为描述方式
实现

```
module samp4_3_1(A, B, C, Y);  
    input A, B, C;  
    output Y;  
    reg Y;  
  
    always@(A or B or C)  
        if(A&B|A&C|A&B&C) //输入变量中  
            //有两个或三个1, 且A为1  
            Y=1;  
        else  
            Y=0;  
endmodule
```

4.3 组合逻辑电路设计

例2 设计一个可以实现4位格雷码和4位二进制编码的相互转换电路，有一个控制端S：当S=1时，可以将输入的4位格雷码转换成4位二进制编码；当S=0时，实现将输入的4位二进制编码转换成4位格雷码。

（一）传统方法

解：（1）确定输入输出

根据题意，有5个输入变量，4个输出变量。

- 5个输入变量分别为控制变量S和四位输入码（格雷码或者4位二进制码）
- 输出变量为四位输出码（4位二进制码或者格雷码）

4.3 组合逻辑电路设计

(2) 列出真值表

控制端 二进制码 格雷码 控制端 格雷码 二进制码

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀	S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	1	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	1	1	0	0	1	1	0	0	1	0
0	0	0	1	1	0	0	1	0	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0	1	0	1	1	0	0	1	0	0
0	0	1	0	1	0	1	1	1	1	0	1	1	1	0	1	0	1
0	0	1	1	0	0	1	0	1	1	0	1	0	1	0	1	1	0
0	0	1	1	1	0	1	0	0	1	0	1	0	0	0	1	1	1
0	1	0	0	0	1	1	0	0	1	1	1	0	0	1	0	0	0
0	1	0	0	1	1	1	0	1	1	1	1	0	1	1	0	0	1
0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	0	1	0
0	1	0	1	1	1	1	1	0	1	1	1	1	0	1	0	1	1
0	1	1	0	0	1	0	1	0	1	1	0	1	0	1	1	0	0
0	1	1	0	1	1	0	1	1	1	1	0	1	1	1	1	0	1
0	1	1	1	0	1	0	0	1	1	1	0	0	1	1	1	1	0
0	1	1	1	1	1	0	0	0	1	1	0	0	0	1	1	1	1

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=0时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	1	1
0	0	1	1	0	0	1	0	1
0	0	1	1	1	0	1	0	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	0	1
0	1	0	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0
0	1	1	0	0	1	0	1	0
0	1	1	0	1	1	0	1	1
0	1	1	1	0	1	0	0	1
0	1	1	1	1	1	0	0	0

本题涉及5个变量，采用卡诺图化简逻辑表达式比较麻烦。为了减少变量个数，可以分为S=0和S=1两种情况来处理。

B_3B_2 B_1B_0		00	01	11	10
		00		1	1
	01			1	1
	11			1	1
	10			1	1
		G_3			

$$G_3 = B_3$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=0时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	1	1
0	0	1	1	0	0	1	0	1
0	0	1	1	1	0	1	0	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	0	1
0	1	0	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0
0	1	1	0	0	1	0	1	0
0	1	1	0	1	1	0	1	1
0	1	1	1	0	1	0	0	1
0	1	1	1	1	1	0	0	0

卡诺图 (Karnaugh Map) 用于函数 G_2 的化简。变量为 B_3, B_2, B_1, B_0 。

B_3B_2 \ B_1B_0	00	01	11	10
00		1		1
01		1		1
11		1		1
10		1		1

图中红色框圈出了 $B_3B_2 = 01$ 和 $B_3B_2 = 11$ 的列，蓝色框圈出了 $B_1B_0 = 01$ 和 $B_1B_0 = 11$ 的列。

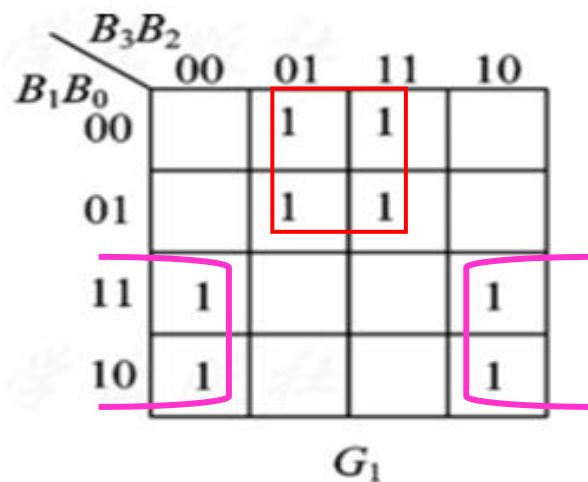
$$\begin{aligned}
 G_2 &= \overline{B_3}B_2 + B_3\overline{B_2} \\
 &= B_3 \oplus B_2
 \end{aligned}$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=0时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	1	1
0	0	1	1	0	0	1	0	1
0	0	1	1	1	0	1	0	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	0	1
0	1	0	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0
0	1	1	0	0	1	0	1	0
0	1	1	0	1	1	0	1	1
0	1	1	1	0	1	0	0	1
0	1	1	1	1	1	0	0	0



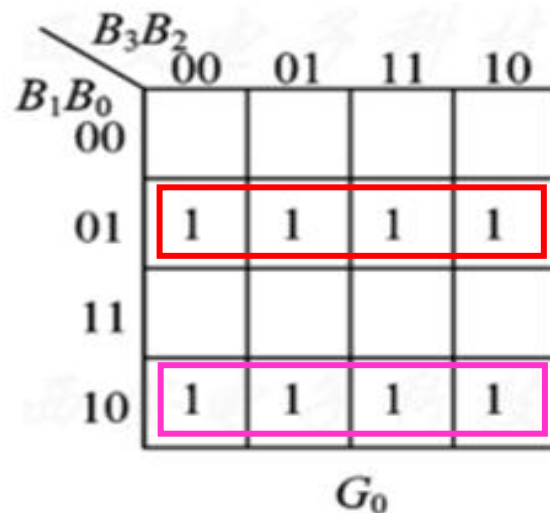
$$G_1 = \overline{B_2}B_1 + B_2\overline{B_1} = B_2 \oplus B_1$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=0时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1	0
0	0	1	0	1	0	1	1	1
0	0	1	1	0	0	1	0	1
0	0	1	1	1	0	1	0	0
0	1	0	0	0	1	1	0	0
0	1	0	0	1	1	1	0	1
0	1	0	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0
0	1	1	0	0	1	0	1	0
0	1	1	0	1	1	0	1	1
0	1	1	1	0	1	0	0	1
0	1	1	1	1	1	0	0	0



$$\begin{aligned}
 G_0 &= \overline{B_1}B_0 + B_1\overline{B_0} \\
 &= B_1 \oplus B_0
 \end{aligned}$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=1时真值表

S	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	0	1	0	0	0	1	1
1	0	1	1	0	0	1	0	0
1	0	1	1	1	0	1	0	1
1	0	1	0	1	0	1	1	0
1	0	1	0	0	0	1	1	1
1	<u>1</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>	0	0	0
1	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	0	0	1
1	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	0	1	0
1	<u>1</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	0	1	1
1	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	1	0	0
1	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>1</u>	1	0	1
1	<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	1	1	0
1	<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>1</u>	1	1	1

B_3B_2					
B_1B_0		00	01	11	10
	00			1	1
	01			1	1
	11			1	1
	10			1	1

G_3

$$G_3 = B_3$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=1时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	0	1	0	0	0	1	1
1	0	1	1	0	0	1	0	0
1	0	1	1	1	0	1	0	1
1	0	1	0	1	0	1	1	0
1	0	1	0	0	0	1	1	1
1	1	1	0	0	1	0	0	0
1	1	1	0	1	1	0	0	1
1	1	1	1	1	1	0	1	0
1	1	1	1	0	1	0	1	1
1	1	0	1	0	1	1	0	0
1	1	0	1	1	1	1	0	1
1	1	0	0	1	1	1	1	0
1	1	0	0	0	1	1	1	1

B_3B_2		00	01	11	10
B_1B_0	00		1		1
	01		1		1
	11		1		1
	10		1		1

G_2

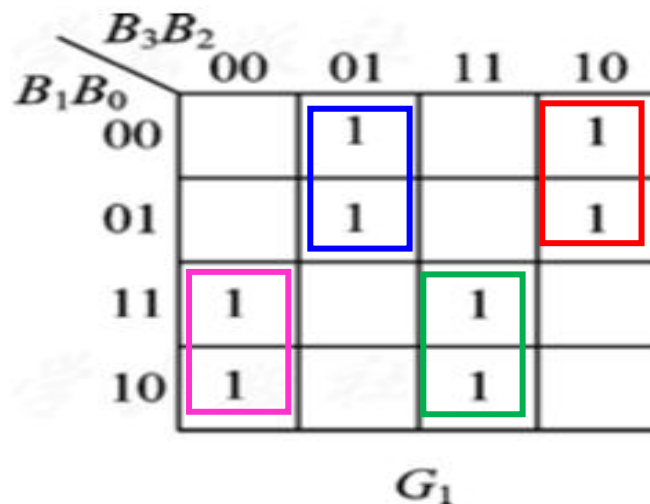
$$\begin{aligned}
 G_2 &= \overline{B_3}B_2 + B_3\overline{B_2} \\
 &= B_3 \oplus B_2
 \end{aligned}$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=1时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	0	1	0	0	0	1	1
1	0	1	1	0	0	1	0	0
1	0	1	1	1	0	1	0	1
1	0	1	0	1	0	1	1	0
1	0	1	0	0	0	1	1	1
1	1	1	0	0	1	0	0	0
1	1	1	0	1	1	0	0	1
1	1	1	1	1	1	0	1	0
1	1	1	1	0	1	0	1	1
1	1	0	1	0	1	1	0	0
1	1	0	1	1	1	1	0	1
1	1	0	0	1	1	1	1	0
1	1	0	0	0	1	1	1	1



$$\begin{aligned}
 G_1 &= \overline{B_3} \overline{B_2} \overline{B_1} + \overline{B_3} \overline{B_2} B_1 + \overline{B_3} B_2 \overline{B_1} + \overline{B_3} B_2 B_1 \\
 &= \overline{B_1} (\overline{B_3} \overline{B_2} + \overline{B_3} B_2) + B_1 (\overline{B_3} \overline{B_2} + \overline{B_3} B_2) \\
 &= \overline{B_1} (B_3 \oplus B_2) + B_1 (B_3 \oplus B_2) \\
 &= B_3 \oplus B_2 \oplus B_1
 \end{aligned}$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

S=1时真值表

S	B ₃	B ₂	B ₁	B ₀	G ₃	G ₂	G ₁	G ₀
1	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	0	1	0	0	0	1	1
1	0	1	1	0	0	1	0	0
1	0	1	1	1	0	1	0	1
1	0	1	0	1	0	1	1	0
1	0	1	0	0	0	1	1	1
1	1	1	0	0	1	0	0	0
1	1	1	0	1	1	0	0	1
1	1	1	1	1	1	0	1	0
1	1	1	1	0	1	0	1	1
1	1	0	1	0	1	1	0	0
1	1	0	1	1	1	1	0	1
1	1	0	0	1	1	1	1	0
1	1	0	0	0	1	1	1	1

		B ₃ B ₂			
		00	01	11	10
B ₁ B ₀	00		1		1
	01	1		1	
	11		1		1
	10	1		1	

G₀

$$\begin{aligned}
 G_0 &= \overline{B_3} \overline{B_2} \overline{B_1} \overline{B_0} + \overline{B_3} \overline{B_2} \overline{B_1} B_0 + \overline{B_3} \overline{B_2} B_1 \overline{B_0} + \overline{B_3} \overline{B_2} B_1 B_0 \\
 &\quad + \overline{B_3} B_2 \overline{B_1} \overline{B_0} + \overline{B_3} B_2 \overline{B_1} B_0 + \overline{B_3} B_2 B_1 \overline{B_0} + \overline{B_3} B_2 B_1 B_0 \\
 &= B_3 \oplus B_2 \oplus B_1 \oplus B_0
 \end{aligned}$$

4.3 组合逻辑电路设计

(3) 根据卡诺图写函数表达式

当 $S=0$ 时:

$$\begin{cases} G_3 = B_3 \\ G_2 = B_3 \oplus B_2 \\ G_1 = B_2 \oplus B_1 \\ G_0 = B_1 \oplus B_0 \end{cases}$$

当 $S=1$ 时:

$$\begin{cases} G_3 = B_3 \\ G_2 = B_3 \oplus B_2 \\ G_1 = B_3 \oplus B_2 \oplus B_1 = G_2 \oplus B_1 \\ G_0 = B_3 \oplus B_2 \oplus B_1 \oplus B_0 = G_1 \oplus B_0 \end{cases}$$

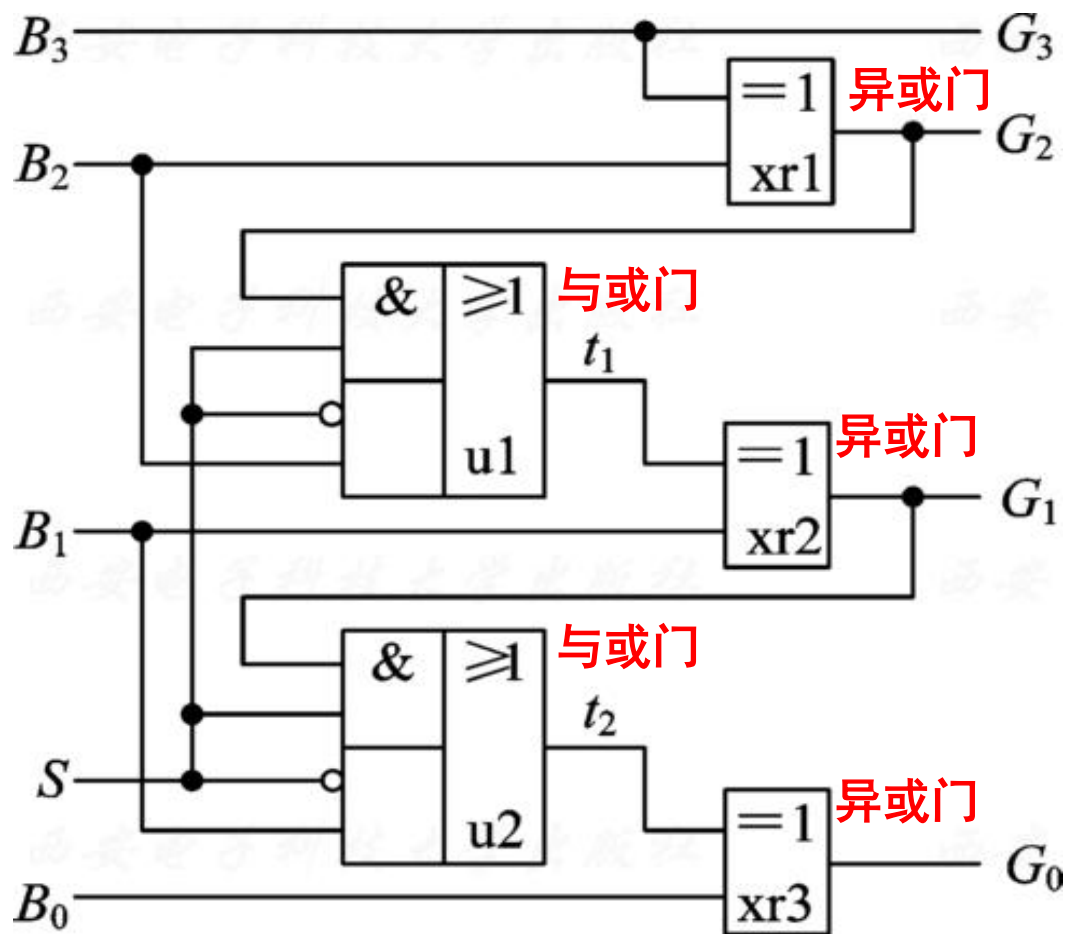


$$\begin{cases} G_3 = B_3 \\ G_2 = B_3 \oplus B_2 \\ G_1 = (\underline{SG_2 + \bar{S}B_2}) \oplus B_1 \\ G_0 = (\underline{SG_1 + \bar{S}B_1}) \oplus B_0 \end{cases}$$

4.3 组合逻辑电路设计

(4) 画出逻辑电路图

$$\begin{cases} G_3 = B_3 \\ G_2 = B_3 \oplus B_2 \\ G_1 = (SG_2 + \bar{S}B_2) \oplus B_1 \\ G_0 = (SG_1 + \bar{S}B_1) \oplus B_0 \end{cases}$$



4.3 组合逻辑电路设计

(二) 编程方法

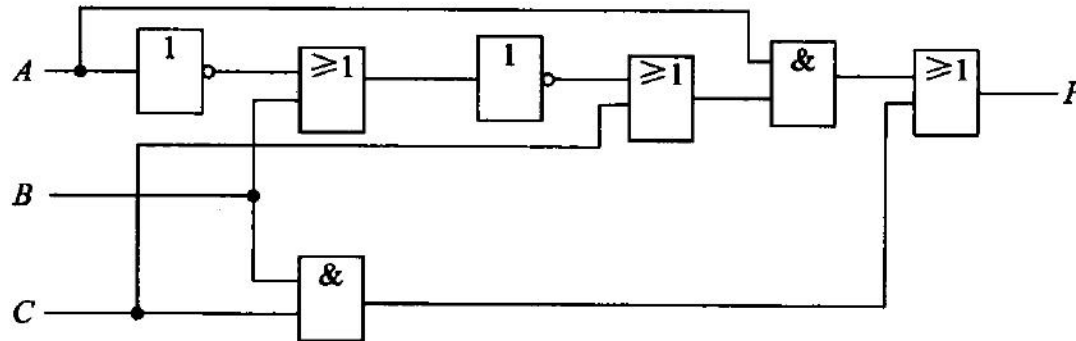
```
module samp4_3_6(B, S, G);  
  input [3:0] B;  
  input S;  
  output reg [3:0] G;  
  always@(S or B)  
  if(S) //格雷码到二进制码的转换  
  begin  
    case(B)  
      4'b0000: G=4'b0000;  
      4'b0001: G=4'b0001;  
      4'b0011: G=4'b0010;  
      4'b0010: G=4'b0011;  
      4'b0110: G=4'b0100;  
      4'b0111: G=4'b0101;  
      4'b0101: G=4'b0110;  
      4'b0100: G=4'b0111;
```

```
      4'b1100: G=4'b1000;  
      4'b1101: G=4'b1001;  
      4'b1111: G=4'b1010;  
      4'b1110: G=4'b1011;  
      4'b1010: G=4'b1100;  
      4'b1011: G=4'b1101;  
      4'b1001: G=4'b1110;  
      4'b1000: G=4'b1111;  
    endcase  
  end  
  else //二进制码到格雷码的转换  
  begin  
    case(B)  
      4'b0000: G=4'b0000;  
      4'b0001: G=4'b0001;  
      4'b0010: G=4'b0011;
```

```
      4'b0011: G=4'b0010;  
      4'b0100: G=4'b0110;  
      4'b0101: G=4'b0111;  
      4'b0110: G=4'b0101;  
      4'b0111: G=4'b0100;  
      4'b1000: G=4'b1100;  
      4'b1001: G=4'b1101;  
      4'b1010: G=4'b1111;  
      4'b1011: G=4'b1110;  
      4'b1100: G=4'b1010;  
      4'b1101: G=4'b1011;  
      4'b1110: G=4'b1001;  
      4'b1111: G=4'b1000;  
    endcase  
  end  
endmodule
```

P167习题2

试写出下图所示组合逻辑电路的输出逻辑表达式，并画出与之功能相同的简化逻辑电路。



P167习题5

设A、B、C为某保密锁的三个按键，当A单独按下时，锁既不打开也不报警，只有当A、B、C，或者A、B，或者A、C分别同时按下时，锁才能被打开；当不符合上述组合状态时，将发出报警信息。试分别用门电路和Verilog模块实现此保密锁电路。

P167习题6

用与非门设计一个四变量表决电路。当变量A、B、C、D有三个或三个以上为1时，输出Y=1，输入为其它状态时，输出Y=0。



武汉大学
Wuhan University

谢谢!

2025-11-20

